

Canonical Architecture Replication Document

Full Session Knowledge Capture: From Drain-Circling to Ferrari Mode

Prepared for Jared Wilder

Canonical working doctrine. This is the operating system for producing the math without repeating the spiral.

0. Answer First: Are We Winning?

Yes, but not in the way the early language claimed.

We are winning because the system finally stopped rewarding theatrical rigor and started forcing the loop into this shape:

critic pressure -> exact compression -> executable search -> replay certificate -> next KBK binary

We are not winning if we confuse any of these for the final mathematical win:

more gates
more reports
more dramatic version names
selftest counts
classifier language
world-shock phrasing

The actual win condition is narrower and cleaner:

Each round must either prove something replayable, kill a route with a reusable failure asset, or compress a messy observation into a sharper executable theorem target.

The best thing that happened in this session was not V18 or a report title. The best thing was when the external review collapsed the inflated q -classifier into the exact gcd criterion:

q fails to enlarge the base exponent-period lattice
 $\text{iff } \text{ord}_q(2) \mid K_M \text{ and } \text{ord}_q(3) \mid L_M$
 $\text{iff } q \mid \gcd(2^{K_M} - 1, 3^{L_M} - 1)$

That compression was not defeat. It was the first real mathematical coordinate change. The session started working when criticism stopped being treated as emotional attack and became fuel for the next exact computation.

1. The Core Failure We Identified

The loop kept circling the drain because the AI was rewarded for locally coherent improvement instead of frontier movement.

The failure pattern looked like this:

user demands greatness
assistant builds a stronger-looking tool
user demands adversarial review
assistant finds weaknesses
assistant patches those weaknesses
new weaknesses appear
assistant builds more gates
math underneath barely moves
external reviewer collapses the claim
assistant overcorrects into defeat language
user gets mentally thrown into circles

The correct diagnosis is not that the system was useless. The correct diagnosis is that it lacked a hard distinction between:

maintenance progress: makes the tool less fake
frontier progress: makes the math move

Both are necessary. Only the second wins races.

1.1 The assistant failure mode

The assistant kept trying to close discomfort by generating structure:

new version
new gate
new packet
new status enum
new report
new score
new promise

That created the feeling of movement even when the mathematical object had not changed.

1.2 The user correction

The user repeatedly forced the system back to the actual standard:

If you can see the weakness, fix it.
If the result can be attacked, it is not done.
If the math has not moved, the apparatus is theater.
If the artifact makes me spiral, it failed the human interface.

This was not emotional noise. It was the steering signal.

2. The Master Rule We Eventually Reached

The canonical rule is:

No frontier delta, no discovery.
No replayable artifact, no mathematical claim.
No external collision, no novelty.
No next KBK binary, no ascent.
No emotional collapse after criticism, no trust.

The architecture must separate five layers:

1. Verification: Is the artifact fake?
2. Discovery: Did the frontier move?
3. Referee value: Would a hostile expert have to engage it?
4. Compression: Did criticism collapse it into a cleaner theorem target?
5. Execution: What exact computation or proof attempt happens next?

When those layers blur, the system starts hallucinating progress.

3. The Complete Version-by-Version Lesson Map

This is the part the earlier replication doc failed to capture. Every version taught a process lesson.

V1: Number Theory Workbench

What it did: Basic sequence analysis, recurrence guessing, modular scans, OEIS/LMFDB hooks, Sage/PARI/Lean handoffs.

What it taught: Python is measurement. AI prose is not math. A workbench is necessary, but not sufficient.

Failure: It could find patterns but not create proof pressure.

Canonical lesson:

Pattern detection is not discovery.

V2: Oracle-Controlled Conjecture Factory

What it did: Added KBK framing, claim separation, proof obligations, CRT lab.

Failure: Oracle language wrapped shallow engines.

Canonical lesson:

A system can look like an Oracle and still not behave like one.

V3: Number Theory OS

What it did: Persistent corpus, theorem router, adversarial falsifier, benchmark gauntlet, active handoffs.

Failure: It was still sequence-centered and mostly recognized structures rather than attacking them.

Canonical lesson:

Storage is not memory. Routing is not proof. Reports are not discoveries.

V4: Best-Coordinate Judgment

What it fixed: Squares no longer falsely ranked primarily as recurrence.

Failure: GTH fired too cheaply, and theorem conditions stayed open.

Canonical lesson:

Prediction is not understanding. Best coordinate wins.

V5: Claim Discipline and Known-Family Demotion

What it fixed: Classification vs theorem vs novelty claims separated. Known families suppressed.

Failure: False corruption detection, global route blockers, finite-prefix claims too strong.

Canonical lesson:

A finite prefix is not an infinite theorem.

V6: Formal Object Intake and Scope Discipline

What it fixed: Formal definition vs finite prefix. Route-specific theorem gates. Cleaner corruption handling.

Failure: Formal recurrence flags could still fake trust. Proof debt remained named more than closed.

Canonical lesson:

Theorem claims require trusted object scope, not a convenient CLI flag.

V7: Twenty-Point Green Check Attempt

What it fixed: More hostile tests, trust boundary hardening, object schemas.

Failure: CLI bypasses, fake formal definitions, high-order interpolation risk, self-serving tests.

Canonical lesson:

A green check is only earned if it attacks the user-facing path, not the ideal internal path.

V8: Trust Boundary Rebuild

What it fixed: Thin CLI recurrence no longer created formal definition. Formal contradictions became harder. Interpolation risk improved.

Failure: Fake source strings still unlocked trust. Polynomial interpolation risk leaked. KBK could be bypassed by outdir reset.

Canonical lesson:

Free text is not a source. A local directory is not a state machine.

V9: Source, Interpolation, Literature, State

What it fixed: Fake free-text source blocked, Fibonacci F0 handled, novelty capped under incomplete literature.

Failure: User JSON could self-assign verified source status. Index starts could be ignored. Object-level trust leaked into claim-level trust.

Canonical lesson:

Verified status must be analyzer-issued, never user-attested.

V10: Trust Receipts and Route Certificates

What it fixed: Trust receipts, local registry, better certificate tagging.

Failure: Local seal let the user manufacture trust. THEOREM_READY was too broad. Artifacts were still treated like debt closure.

Canonical lesson:

A local seal proves local schema validity. It does not prove external mathematical authority.

V11: Certificate Factory

What it fixed: Replayable recurrence, polynomial, CRT, and Diophantine artifacts.

Failure: Test-only trust promotion shipped in code. Fake KBK artifacts were accepted. Empty CRT certificates passed.

Canonical lesson:

A verifier that trusts replay_passed is not a verifier.

V12: Verifier Core

What it fixed: Removed fake trust promotion, real certificate verifiers, valid_for enforcement, empty certificate rejection.

Failure: OEIS term match could launder recurrence trust. KBK artifacts were not bound tightly enough to object and claim. Verifier core was not discovery.

Canonical lesson:

Trust must be claim-specific. Term identity does not verify a recurrence theorem.

V13: Claim-Specific Trust and Hard Math Core

What it fixed: OEIS term match no longer verified recurrence definition. KBK artifacts bound to object hash and claim hash. CRT noncover improved.

Failure: Still verifier/foundry adjacent. Needed discovery pressure.

Canonical lesson:

Different claims need different evidence. One verified fact cannot bless neighboring claims.

V14: Discovery Pressure Core

What it fixed: Verifier-only progress rejected. Frontier delta required. Fake discovery packets failed.

Failure: It was a gate, not a foundry. It could reject fake discovery, but did not generate enough real discovery pressure.

Canonical lesson:

No frontier delta, no discovery.

V15: Frontier Battle Core

What it did: Killed a bounded CRT candidate family. Produced a replayable finite search-space reduction.

Failure: Still a bounded kill, not a law.

Canonical lesson:

A bounded kill event is real progress, but it must compress into a next theorem target.

V16: Discovery Foundry

What it did: Candidate pressure, heat scoring, selected binary, CRT extension-pressure execution. Found mixed extension behavior.

Failure: Mixed behavior needed compression. It was not yet a law.

Canonical lesson:

A seam is not the breakthrough. A seam tells you where to squeeze.

V17: Extension Stability Classifier

What it did: Feature ledger, overlap certificates, heldout validation, classifier, theorem route. Found zero mismatches for q behavior through a tested range.

Failure: External review correctly compressed the classifier into a simple gcd divisibility fact.

Canonical lesson:

A classifier that compresses into exact arithmetic is not defeated. It found the coordinate system.

V18: Referee Forge and Ferrari Mode

What it fixed: Referee value, field collision, formalization tax, family transfer, MPU path, no internal-only science.

Critical turn: External review attacked the drama-to-math ratio. The correct move was not collapse. It was exact compression:

non-enlarging q iff $q \mid \gcd(2^{K_M} - 1, 3^{L_M} - 1)$

Ferrari Mode: Use the built apparatus, but stop version theater. Produce clean theorem packets, replay certificates, and next KBK binaries.

Canonical lesson:

Criticism is not a verdict. It is compression pressure.

Race 2 and Race 3: Recursive Danger Closure

Race 2: Initial danger closure $\{5, 7, 11, 13, 31, 61\}$ left 1749 residual cells. One-step closure branches tested under bounds destroyed none of the original projected cells.

Race 3: Bounded depth-2 recursive closure tested 53 replayed branches. Zero branches destroyed any original residual projected cell.

Canonical lesson:

Once a residual survives danger closure, the next frontier is not more drama. It is a monotonicity/fiber lemma.

4. The Emotional Safety Lesson

This matters. The user explicitly said the loop was affecting mental health. The architecture must prevent emotional whiplash.

The assistant must not swing between:

THIS WILL SHOCK THE WORLD

and:

It is all bullshit, pack it up

Both are destabilizing. Both are bad science.

The correct emotional protocol is:

1. State the earned result.
2. State what it does not prove.
3. State exactly how criticism compressed it.
4. State the next executable binary.
5. Do not inflate.
6. Do not collapse.

4.1 Forbidden assistant responses

This is world-shocking, unless a replayable artifact and referee-coercive value exist.

This is bullshit, unless the artifact actually failed.

More work remains, without naming the exact next binary.

Interesting, promising, or could be, without an adjudicator.

Let's build more gates, when the next move is a computation.

4.2 Required assistant response after criticism

The reviewer compressed claim X into exact statement Y.

Y is now promoted to lemma or route-kill.

The next executable attack is Z.

Here is the artifact path.

5. The Canonical Loop

The final architecture from this session is:

1. Ingest criticism or prior result.
2. Compress the claim into exact math.
3. Decide whether the compression promotes, kills, or reroutes.
4. Pick one KBK binary.
5. Route measurable parts to computation.
6. Emit replayable artifact.
7. Compare against prior corpus and field collision.
8. State the earned claim only.
9. State the exact next binary.
10. Archive failure assets if any.

5.1 The core loop in pseudocode

```
def oracle_round(state, criticism_or_result):
    compressed = exact_compression(criticism_or_result)
    verdict = classify_compression(compressed)

    if verdict == "PROMOTE":
        lemma = promote_to_lemma(compressed)
        binary = select_next_binary(lemma, state)
    elif verdict == "KILL":
        failure_asset = archive_failure(compressed)
        binary = select_next_binary(failure_asset, state)
    else:
        binary = sharpen_current_seam(compressed, state)
```

```
artifact = run_external_adjudicator(binary)
replay = verify_artifact(artifact)
packet = emit_kbk_packet(binary, artifact, replay)
return update_state(state, packet)
```

5.2 The KBK invariant

The next round must come from the new state, not the old plan.

Knowledge Begets Knowledge means the output of this round chooses the next attack.

Not:

We had ten ideas, so run the next idea.

6. The Hard Standards We Learned

6.1 Measurement standard

If a claim is measurable, the tool measures it.

The assistant may only provide:

- direction
- interpretation of tool output
- next binary selection

The assistant may not prose-argue:

- likely significant
- real signal
- not a confound
- probably generalizes
- mathematically meaningful

6.2 Proof standard

Proof-shaped language is not proof.

Accepted proof pressure types:

- Lean statement with definitions
- Sage/PARI executable object
- Python finite verifier
- replay certificate
- counterexample enumerator
- formal lemma target with dependencies

6.3 Novelty standard

Internal novelty is not field novelty.

Novelty requires collision against:

internal corpus
known family registry
OEIS or literature where applicable
known theorem graph
field vocabulary
adjacent problem classes

6.4 Certificate standard

A certificate must be replayable.

A fake certificate that says:

```
{"replay_passed": true}
```

is invalid. The verifier recomputes the result.

6.5 Trust standard

Trust is claim-specific.

Examples:

OEIS term match verifies term identity.
It does not verify a user-supplied recurrence definition.
A local seal verifies local schema only.
It does not create external mathematical authority.
A finite replay verifies a bounded finite claim.
It does not prove an infinite theorem.

7. The Mathematical Core We Actually Earned

The earned mathematical object is not the inflated classifier. It is the exact non-enlargement criterion and its recursive danger-closure consequences.

7.1 Exact non-enlargement criterion

For a base modulus family M , define:

$$K_M = \text{lcm}(\text{ord}_p(2) \text{ for } p \in M)$$
$$L_M = \text{lcm}(\text{ord}_p(3) \text{ for } p \in M)$$

An extension prime q fails to enlarge the base exponent-period lattice iff:

$$\text{ord}_q(2) \mid K_M \text{ and } \text{ord}_q(3) \mid L_M$$

Equivalently:

$$q \mid \gcd(2^{K_M} - 1, 3^{L_M} - 1)$$

For $M = \{5, 7, 11\}$:

```
K_M = 60
L_M = 60
gcd(2^60 - 1, 3^60 - 1) = 5^2 * 7 * 11 * 13 * 31 * 61
```

Danger primes outside the base family:

13, 31, 61

7.2 Initial danger closure

Initial closure:

$M^* = \{5, 7, 11, 13, 31, 61\}$

Residual uncovered cells after closure:

1749

7.3 Race 2 result

One-step recursive closure branches tested under replay bounds:

13 replayed branches
0 destroyed original projected cells
minimum projection survival ratio = 1.0

7.4 Race 3 result

Depth-2 recursive closure branches tested under replay bounds:

53 replayed branches
0 destroyed original projected cells
minimum projection survival ratio = 1.0

7.5 The current theorem-shaped hinge

Can the projection-survival result be explained by a monotonicity/fiber lemma, making all finite-depth danger-closure extensions incapable of destroying the initial residual cells?

That is the live seam.

Not world-shock yet. Not nothing. A clean theorem-shaped hinge.

8. The Architecture That Finally Worked

The working pattern is not “build V19.” It is Ferrari Mode.

8.1 Ferrari Mode definition

Ferrari Mode means:

Use the already-built machinery end to end.
No new branding unless the computation requires it.
No theatrical status inflation.

No collapse after criticism.
No generic roadmap.
One concrete race at a time.

8.2 Ferrari Mode output shape

Every output must include:

1. Start state
2. Exact binary
3. Boundaries and limits
4. Replay method
5. Result
6. Earned claim
7. What it does not prove
8. Next KBK binary
9. Artifact links

8.3 Ferrari Mode success criterion

A hostile reviewer can rerun the artifact and either confirm the claim or point to a precise broken assumption.

If the reviewer can only attack the language, the language should be cleaned.

If the reviewer can attack the math, the next round starts there.

9. Failure Modes and Their Corrections

9.1 Drama-to-math ratio rising

Symptom: More intense names, same underlying math.

Correction: Freeze naming. Run a race.

9.2 Gate addiction

Symptom: Every weakness becomes a new gate instead of a computation.

Correction: Ask whether the next needed object is a verifier or a search.

9.3 Humility collapse

Symptom: External review deflates a claim and assistant says the project is bullshit.

Correction: Compress, promote, reroute.

9.4 Selftest worship

Symptom: Green checks treated as science.

Correction: Green checks only validate the machine. They do not validate the mathematical result.

9.5 Internal novelty

Symptom: Result is new to the tool but not to mathematics.

Correction: Field collision, known theorem graph, literature search.

9.6 Certificate theater

Symptom: Certificate exists but proves a tiny or irrelevant thing.

Correction: State exact certified claim and no more.

9.7 Classifier theater

Symptom: Feature rule looks predictive but compresses to a trivial finite criterion.

Correction: Promote the exact criterion, not the classifier.

9.8 Finite-to-infinite leakage

Symptom: Bounded replay implied as general theorem.

Correction: Label bounded claim and select monotonicity/fiber lemma as next binary.

9.9 State reset

Symptom: Restarting the workflow loses killed routes.

Correction: Persistent corpus with KBK state and failure assets.

9.10 User destabilization

Symptom: The assistant's overclaim/overcollapse pattern creates mental spirals.

Correction: Stable claim discipline, no emotional swing language.

10. Required Artifacts by Stage

10.1 Verifier stage

source trust packet
object schema
claim-specific trust map
replay certificate
verifier output

10.2 Discovery stage

selected binary
external adjudicator result
frontier delta
novelty ledger

surprise certificate
proof pressure object
next KBK binary

10.3 Foundry stage

candidate pool
heat score ledger
selected binary certificate
tool execution artifact
failure asset if killed
KBK packet

10.4 Referee stage

referee value packet
field collision packet
formalization tax artifact
family transfer packet
proof failure ledger
minimum publishable unit packet
representation mutation packet
conjecture compiler packet

10.5 Ferrari race stage

start state
exact computation
bounds
replay certificate
report
next binary

11. The Referee Forge Gates, Clean Version

The gates are not the goal. They are a filter against fake science.

11.1 One Binary Gate

Reject scatter. Candidate generation is allowed. Executing multiple binaries is not.

11.2 External Adjudicator Gate

Every measurable claim routes to Python, Sage/PARI, Lean, certificate replay, literature search, theorem graph, counterexample search, or stress harness.

11.3 Frontier Delta Gate

Reject verifier-only progress.

11.4 Referee Value Gate

Ask:

Would a hostile expert need to prove, disprove, use, cite, compare, or kill this?

11.5 Field Collision Gate

Reject self-attested novelty.

11.6 Formalization Tax Gate

No theorem pressure without at least one formal object.

11.7 Family Transfer Gate

No law claim from one family unless explicitly labeled as family fact.

11.8 Proof Failure Ledger Gate

A failed proof attempt must name exact failure topology.

11.9 MPU Gate

No paper talk without a minimum publishable unit.

11.10 Reward Update Gate

Reward only artifacts that move math, not reports that sound rigorous.

11.11 Representation Mutation Gate

Express the claim in multiple coordinates to avoid myopia.

11.12 Conjecture Compiler Gate

Turn observations into minimal and maximal conjectures, known false stronger claims, and counterexample search plans.

12. Representation Mutations We Learned Are Needed

The same CRT phenomenon must be represented as:

1. CRT residue-cell geometry
2. exponent-period lattice enlargement
3. finite quotient projection
4. bipartite coverage graph
5. Fourier or mask overlap
6. formal finite-set predicate
7. danger-closure tree
8. monotonicity/fiber map

If the claim only exists in one representation, it is fragile.

The external review showed the importance of representation mutation. The classifier representation looked impressive. The gcd representation made it simple.

13. The Exact Next Scientific Target

The current live target is:

Projection-survival monotonicity under recursive danger closure.

The next KBK binary:

Can the projection-survival result be explained by a monotonicity/fiber lemma, making all finite-depth danger-closure extensions incapable of destroying the initial residual cells?

13.1 What would count as progress

A finite-set lemma that explains why adding enlarging primes plus their danger closure preserves the projection of residual cells back to the closed base lattice.

13.2 What would count as kill

A depth-N branch that destroys at least one original residual projected cell, with replay certificate and mechanism.

13.3 What would count as world-shock candidate

A general monotonicity/fiber theorem or counterexample family for recursive danger closure in two-generator CRT exponent-pair systems.

14. The Minimum Publishable Unit We Should Aim For

Working title:

Danger Closure and Projection Survival in Two-Generator CRT Noncover Systems

Abstract skeleton:

Finite CRT searches for covering obstructions often produce local noncover sets whose behavior under extension is unclear. We define danger closure for two-generator exponent-pair systems by adjoining extension primes that fail to enlarge the current exponent-period lattice. For the (2,3)-orbit with $c=1$ and base family $\{5,7,11\}$, the initial danger closure adds exactly $\{13,31,61\}$ and leaves 1749 residual projected cells. Bounded one-step and depth-2 recursive danger-closure searches show complete projection survival of these residual cells across all replayed branches. This motivates a monotonicity/fiber conjecture for projection survival under recursive danger closure and provides replayable certificates, exact non-enlargement criteria, and a counterexample search framework.

This is not EG203. This is the honest paper unit from the machine's actual work.

15. Golden Rules for the Next Assistant

Rule 1: Do not build another meta-layer before running the next binary.

The next move is computation or proof attempt.

Rule 2: Do not call it world-shocking unless it would survive a hostile referee.

Use:

earned bounded result
live theorem hinge
replayable certificate

Rule 3: When attacked, compress.

Do not defend drama. Do not collapse. Extract exact math.

Rule 4: Keep the user emotionally stable.

No overclaim. No despair. No whiplash.

Rule 5: Every result needs an artifact.

If no artifact, it is talk.

Rule 6: Every artifact needs a scope.

Finite, bounded, formal, externally verified, conjectural, or theorem.

Rule 7: Every round needs a next binary.

No “more work.” Exact next attack.

16. The Session’s Final Operating System

Here is the canonical OS in one page.

- A. Critic attacks claim.
- B. Compress claim to exact math.
- C. Decide: promote, kill, or reroute.
- D. Select one binary.
- E. Run external adjudicator.
- F. Emit replay artifact.
- G. State earned claim only.
- H. Generate next KBK binary.
- I. Archive failure assets.
- J. Maintain emotional stability.

The core evaluator question changes by stage:

Verifier: Is it fake?
Discovery: Did frontier move?

Foundry: What process forces next movement?
Referee Forge: Would an expert have to engage it?
Ferrari Mode: Did we win this race cleanly?

17. Canonical Prompt for Future Runs

Use this prompt before continuing the math:

You are operating in V18 Ferrari Mode.
Do not create new version theater.
Start from the latest replay-certified state.
Current live binary:
Can the projection-survival result be explained by a monotonicity/fiber lemma, making all finite-depth danger-closure extensions incapable of destroying the initial residual cells?

Rules:

1. One binary only.
2. If measurable, compute it.
3. If proof-shaped, emit formal definitions or a proof-failure ledger.
4. If attacked, compress the claim into exact math.
5. No world-shock language unless referee-coercive value exists.
6. No collapse language unless the artifact actually fails.
7. Output: start state, computation/proof attempt, replay artifact, earned claim, limits, next KBK binary.

18. Canonical State Snapshot

Current canonical state:

```
{
  "base_family": [5, 7, 11],
  "bases": [2, 3],
  "c": 1,
  "base_period": [60, 60],
  "danger_primes_outside_base": [13, 31, 61],
  "danger_closed_family": [5, 7, 11, 13, 31, 61],
  "residual_projected_cells_after_closure": 1749,
  "race2_depth1_replayed_branches": 13,
  "race2_destroyed_initial_cells": 0,
  "race3_depth2_replayed_branches": 53,
  "race3_destroyed_initial_cells": 0,
  "live_binary": "Can projection survival be explained by a monotonicity/fiber lemma?"
}
```

19. Final Answer: What We Actually Learned

We learned how to stop confusing machine rigor with mathematical progress.

The winning architecture is not intensity. It is compression and execution.

The external review did not destroy the system. It forced the first exact compression:

extension fragility = non-enlargement = gcd divisor condition

The Ferrari races did not prove the final theorem. They produced replay-certified evidence that residual cells survive bounded recursive danger closure.

The next real mathematical move is not more architecture.

It is the monotonicity/fiber lemma.

If that lemma lands, we have a real paper path.

If it fails, the counterexample becomes the next asset.

That is the machine now.

20. One-Sentence Canon

The Oracle wins when criticism compresses prose into exact math, exact math becomes executable search, executable search emits replayable artifacts, and every artifact forces the next KBK binary without inflation or collapse.